UBS VERIFICATIONS · DECISION ENGINE · JULY 2026

The Verifications Decision Engine

2.75 million replayed case-days — and the learning operation that evidence can unlock. Five pieces: the business case, the ceiling of today's automation, one case through the machine, the engineering, and the philosophy. Nothing sent without a person — yet: automation is a later, earned stage.

Reading notes. All figures are backtested and as-of (no future information); correlations are observational — no channel or wording is claimed to cause outcomes until the pilot measures it; the system is decision support today — nothing auto-sends yet; automation is designed in as a later, evidence-gated stage. The planned next layers: an AI contact researcher that finds fresh, verified contact paths, and a small language model that drafts the emails, faxes, notes, and call scripts — sequenced last, behind the causal instrumentation that can grade them.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

HOW TO READ THIS PACKET

The argument in five parts

1	**The executive briefing**	establishes the business problem, the working foundation, and the destination
2	**The Murphy Brown field report**	shows why scripted execution needs an operating layer around it
3	**The illustrated walkthrough**	follows one case through that layer — today, and in the next stage
4	**The engineering deep dive**	shows how the system is built, tested, and extended
5	**Earned autonomy**	explains why each increase in authority follows evidence from the stage before it

One sentence carries all five: six machine-learning models find the winnable work, the operating system makes acting on it safe, the pilot teaches it what works — and each completed case makes the next decision better.

A Wright Original · Built by **SNH Strategic Initiatives**

EXECUTIVE BRIEFING

The Verifications Decision Engine

People have judgment, but the queue has no strategy. Automation has speed, but it has no orchestration. The Verifications Decision Engine gives both lanes a governed plan — and turns every outcome into evidence for the next case.

Problem people have a queue; automation has a touch; neither has a complete case strategy

Evidence 2.75 million historical case-days replayed

Status built and backtested; next step is live integration with human review

The operation has two lanes — and one missing layer

Verifications currently operates through two lanes: people working queues and automation executing individual touches. Both produce real work. Neither is supported by a common decision system that reads the case, explains what matters, and governs what should happen next.

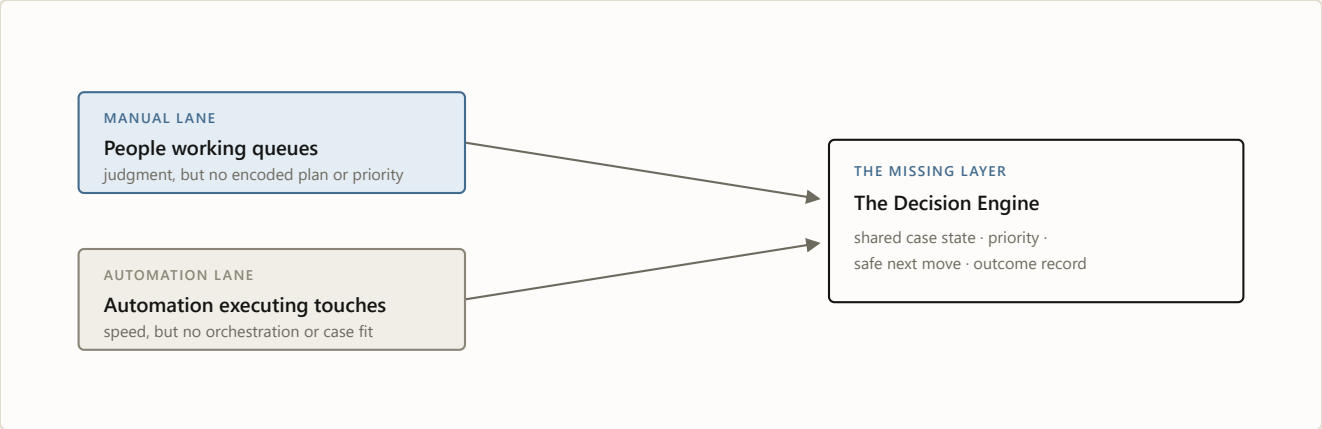
In the manual lane, people receive a queue — not a strategy. The worklist does not consistently tell them which cases are most recoverable, which are beginning to stall, which have already received a reply, or why one case should be worked before another. Agents bring their own experience and judgment, but the system makes them reconstruct the plan case by case. The operation therefore depends on people to supply the prioritization, memory, and orchestration that the workflow itself does not provide.

The automation lane has the opposite strength and the same underlying weakness. Automation can execute a touch quickly when the contact path and next action are already clear. But it does not consistently decide whether the case is a good fit, whether another touch is allowed, when the path should change, what state the case should return in, or what the outcome should teach us. That produces fast cases when the script fits — and downstream handbacks, repeated work, reopening closures, and human reconstruction when it does not. *The ceiling of scripted automation*, the Murphy Brown field report, documents that pattern in detail.

The blended "unable to verify" number reflects the same weakness. One in five worked cases ends unable to verify, but the result does not tell leadership whether the case was impossible, the contact path failed, or a recoverable case received the wrong next move. Different problems enter similar workflows and disappear inside the same outcome.

The common problem is not a lack of effort or automation. It is the absence of an intelligence and orchestration layer shared by both. People have judgment without systematic decision support. Automation has speed without case-level judgment. Neither lane produces the complete record needed to learn what works.

That is the problem the Verifications Decision Engine was built to solve.



Neither lane fails for lack of effort. Both are missing the same layer — the one the Decision Engine supplies.

The original ask was a learning operation, not just a model

The original brief asked how quickly viable small language models could be produced from our own casework, whether manual entry could be reduced by 90% or more, and whether the

operation could learn which communication works well enough to automate it.

That is not only a language-model question. Before a system can responsibly write or send the next communication, the operation must tell it what is true, what is allowed, what has already been tried, and whether the last communication worked.

So I started one layer below the language model: build the decision and evidence system that makes the language layer viable.

The specialist prediction models running today are not the future SLM. They are the decision layer that tells the future SLM when to act, what evidence it may use, and how its work will be judged. The first specialist forecasts were working in one week and materially improved within the next two — a demonstration of how quickly useful machine learning can be built on our casework; the language layer remains the next phase.

What we built: intelligence plus control

MACHINE-LEARNED INTELLIGENCE

a fleet of specialist models · reads every open case each morning

Different specialists estimate different parts of the case's path: whether it is likely to resolve quickly, whether it is beginning to stall, and whether it is likely to verify at all.

The models do not authorize actions. They provide an informed read of where the case is heading and which eligible move deserves attention first.

THE OPERATING LAYER

deterministic · names state, removes, ranks, records

It names the case's current state, removes actions that are prohibited or no longer relevant, ranks what remains, presents one recommendation to a person, and records the decision and outcome.

The models answer what is likely. The operating layer answers what is true and what may happen. A person decides what happens now.

Intelligence without the operating layer produces recommendations the operation cannot safely use. Rules without intelligence produce safe but indiscriminate workflows. The value comes from the handoff between them.

Every case follows the same handoff

1 · Evidence shows what is known	The morning record establishes what is currently known about the case.
2 · The model fleet predicts where it is heading	Specialists estimate where the case is heading — resolve quickly, begin to stall, or never verify.
3 · Rules name the state what is true now	The operating layer decides what is true now — replied, blocked, cooling down, workable.
4 · Rules remove; models rank eligible moves, ordered	Prohibited or premature actions leave the table; the eligible choices that remain are ordered.
5 · A person decides accept or override	The recommendation is accepted or overridden — and the override is itself a signal.
6 · The outcome teaches into the record	The decision, communication, reply, and final result enter the learning record.

The model never receives authority to restore an action the rules removed. That is how prediction can improve the work without owning compliance.

What 2.75 million historical case-days proved

The operation can see difficulty before spending the effort. At intake, the system separates likely successes from likely failures with approximately 81% sorting power. Across the full daily population, the specialist forecasts remain useful as cases age and the obvious successes leave the queue.

The blended UTV number hides radically different cases. The easiest predicted tenth of cases fails about 3% of the time. The hardest tenth fails about 48% of the time. That 16-fold

spread makes a fairer operating measure possible: evaluate outcomes against what was realistically achievable and direct effort toward cases where intervention can still matter.

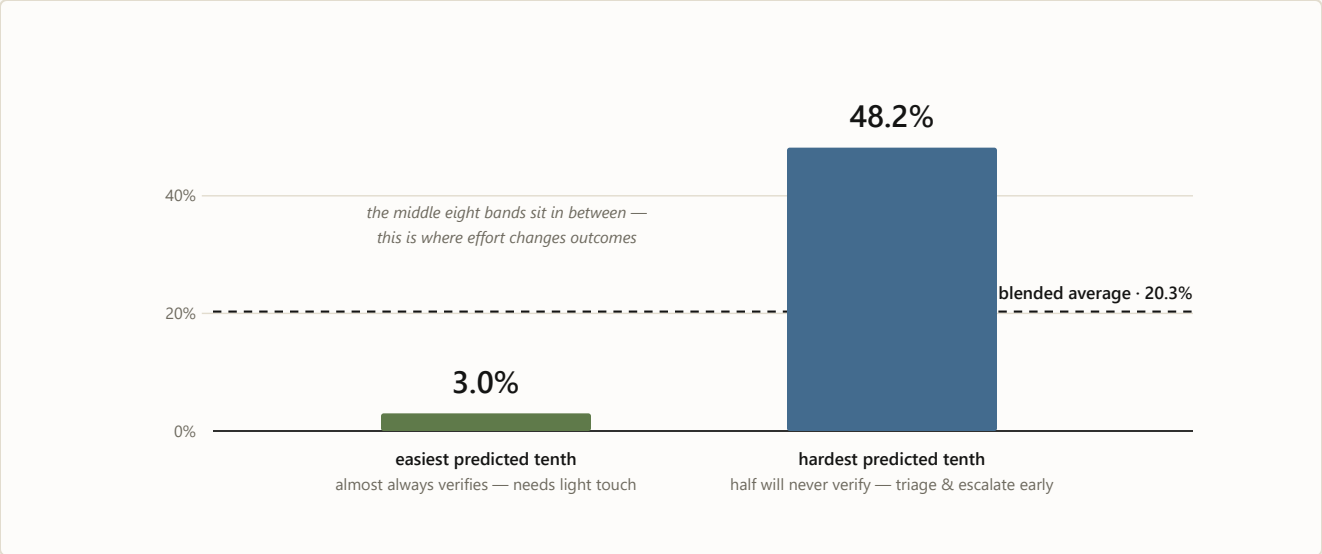


CHART 3 Observed never-verify rate by predicted difficulty band (May 2026 holdout): the easiest tenth 3.0%, the hardest 48.2% — the 16-fold spread that makes a difficulty-adjusted scoreboard possible.

A simple rule found substantial avoidable work. On roughly 400,000 replayed case-days, a response was already on record while the existing flow would have continued contacting the party. The operating layer routes those cases to review before another touch can occur. At an illustrative three minutes per avoided touch, that measured count represents about 20,000 hours, or ten operator-years, of motion. The touch count is measured; the three-minute assumption is not. Live logs replace the assumption with actual handle time.

The two-part design held its safety boundary. Across 2.75 million historical case-days, the replay produced zero do-not-contact and hard-policy violations. That result did not depend on the models being perfect. The models never received authority to override the rules.

These are historical replay results, not live-pilot outcomes. They establish that the system can read, route, and constrain the work. Integration establishes the live operational effect.

For people, the queue becomes a plan

On the first day of integration:

- Every open case has a named state, reason, and next move.
- Worklists reflect difficulty and urgency rather than arrival order alone.
- A reply on record prevents another outreach attempt.
- A prohibited action is removed before ranking.
- A person can accept or override the recommendation.
- Every decision begins contributing to the learning record.

The first release does not replace the operator. It gives the operator a clearer case, a safer set of choices, and a better-aimed first hour.

For people, a queue becomes a plan. For automation, a touch becomes one governed step inside a complete case strategy.

The first release recommends. The destination learns.

Once the live record connects each recommendation to the action, message, delivery, reply, and final outcome, the operation can begin learning the complete play — not merely which case looks difficult, but which channel, timing, contact path, and message should be used next.

That record unlocks the rest of the original ask. An AI contact researcher repairs weak contact paths, and a small language model drafts the outreach — emails, faxes, notes, call scripts. People approve the work first. Narrow, stable lanes earn controlled auto-send only from their own logged results.

The long-term product is not a model or a bot. It is a verification operation that improves after every completed case.

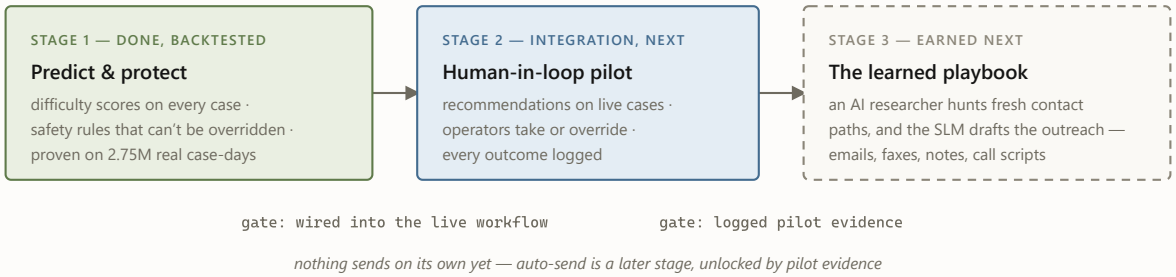


CHART 4 Now, next, then — each stage gated by the evidence the one before it produces. Stage 3, the part every vendor promises on day one, is deliberately last: it gets proven rather than claimed.

What leadership is being asked to do

- **Adopt difficulty-adjusted UTV as the operating scoreboard.** Measure misses against what was realistically achievable, not only against the blended average.
- **Integrate the Decision Engine beside the live team with complete outcome logging.** Recommendations remain under human review while the operation measures adoption, overrides, turnaround, and outcomes.
- **Use the resulting evidence to sequence the larger AI build.** Contact research, SLM drafting, and controlled automation advance only from the record produced by live casework.

The immediate decision is not whether to let AI run verifications autonomously. It is whether to put the decision and evidence layer into the workflow so the operation can improve now — and so the larger AI ambition can be built on proof rather than promise.

That is how the original SLM ambition becomes viable: not language first, but a system that knows what it is automating and whether it worked. ◆

The fine print, kept honest. All results are historical replay/backtest results using only information available as of each case-day; no live-pilot effect is claimed. Sorting-power figures are translated from the validated model metrics

documented in the Engineering deep dive. Historical channel and message patterns remain observational until live outcome logging captures recommendation, action or override, message version, delivery or reply, and final outcome. Nothing auto-sends today, and no decision is finalized by a machine.

Deeper reading: the Murphy Brown field report (the problem this system closes) · the illustrated walkthrough (follow one case through the machine) · the engineering deep dive · the design philosophy.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · Executive briefing · July 2026

A Wright Original · Built by **SNH Strategic Initiatives**

FIELD REPORT

The ceiling of scripted automation

Murphy Brown, our verification bot, is genuinely fast when the contact path is already clear — it fully contained about 44.5% of the tens of thousands of cases it touched over four months. The other half flowed back to people with nothing but a note, and its closures are reopening at ten times the winter rate. What the ops deep-dive actually found, why the answer is to govern the bot rather than retire it — and how it becomes the execution layer inside the decision engine.

Evidence window Mar 1 – Jun 30, 2026 (live warehouse refresh)

Claim posture association language only — see the fine print

THE 60-SECOND VERSION

- **What works:** the bot fully contains ~44.5% of what it touches, usually within a day or two. Where the contact path is already laid and the case fits the script, it executes fast — genuinely.
- **What breaks:** the other 55.5% flows back to people as note-only handoffs; the closures it does make are reopening at ten-plus times the winter rate; and its handbacks queue behind fresh work (EDUV +30.6h).
- **Why it happens:** nothing gates what it picks up, nothing owns the state it leaves behind, nothing locks what it closes, and nothing records enough to prove any of it — so humans became the control layer. Not bugs: the ceiling of scripted automation.
- **What to do:** keep the bot; add the operating layer. Five asks — closure lock, triage gate, handback re-rank, one holdout config rule (the causal number lands ~July 15 either way), and the structured payload. The decision engine is that layer; the bot becomes a governed channel inside it.

Five numbers tell the story:

55.5%

Of bot-touched cases flow back to people

Roughly 14,800 of 26,700 — about four downstream human attempts apiece, 64,000+ rows in all.

~43%

Of the bot's closures now reopen after the result is sent

Up from ~3–4% in February. June figures are floors, not finals.

+30.6h

Extra wait for an EDUV handback vs. fresh work

Not a parking defect — 99.98% are agent-assigned. A queue-ranking problem.

1.7%

Containment in the worst tenth of what it picks up

Its wins were predictable before pickup (77% sorting power) — nobody was gating the door.

~45%

Of eligible tickets picked up by neither automation

About 113,000 EMPV/EDUV tickets where automation is live — the gate fails in both directions.

Source: the consolidated MB/SNH-AI findings (2026-06-30) and the July 2–3 forensics one-pager, read-only warehouse analysis. All figures are the current numbers of record for their stated windows.

A two-speed workflow

When MB touches a case, one of two things happens. About 44.5% of the time it fully contains the case — closed in one or two days, no human ever needed, genuinely fast. The other 55.5% of the time the case flows back to a person. That split, not any single defect, is the operating reality:

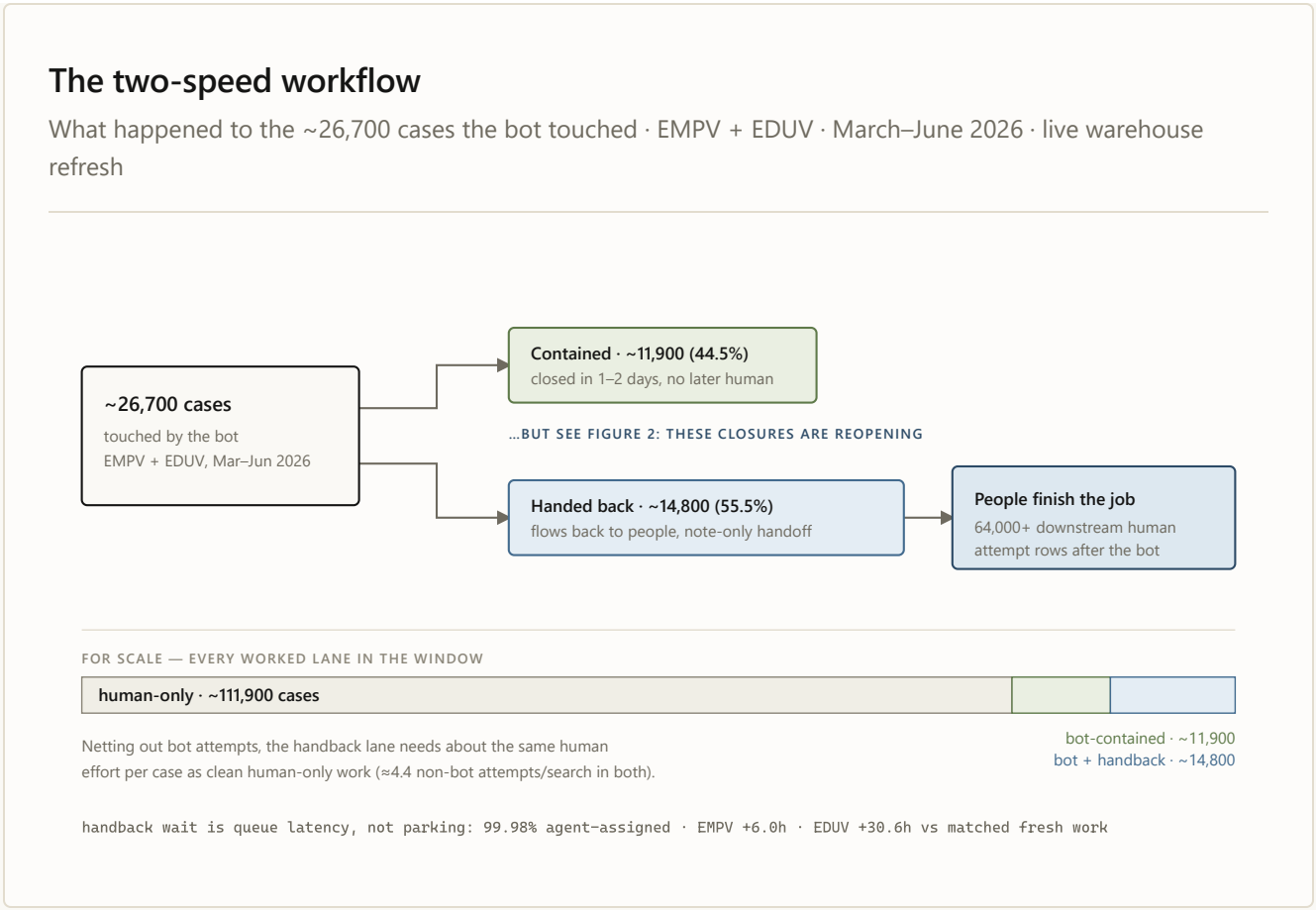


FIGURE 1 The two-speed workflow. The matched comparison is honest here: handback cases don't take clearly *more* human effort than comparable human-only cases — the durable finding is that the bot leaves a large residual queue with weak fast-closeout performance and no machine-readable handoff, so people must rediscover each case before continuing it.

Fairness requires the other half: **when the bot contains a case, it is genuinely fast.** The contained lane completes in about 61 hours versus 161 for human-only work, with a higher raw success rate (80.5% vs 73.7%). Two caveats keep that honest: these are raw lane averages, not matched comparisons — the bot picks up easier-looking work — and the clocks flatter it, because the bot fires instantly while human work queues first. Re-anchored at first touch, EMPV's speed advantage halves to −37.6 hours and EDUV flips to 23.7 hours *slower* than fresh human work. The containment is still real — which is exactly why the recommendation is to govern the bot, not retire it.

And there is a third speed the two-lane picture can't show: **never picked up at all.** Where automation is live, 44.9% of eligible EMPV/EDUV tickets — about 113,000 — were touched by neither the bot nor the SNH-AI engine behind it (measured per ticket). Part of the mechanism is already known: all-or-nothing API routing pushed about 50,000 excluded-client searches back to

manual work, a modeled 7.2–8.6k added manual searches per month that still needs configuration history to be proof-grade. The gate fails in both directions — the bot picks up cases it can't contain (its worst decile contains 1.7%) and never reaches nearly half of what it could. One gating layer fixes both.

And the contained lane — the bot's success story — has a quality problem growing inside it:

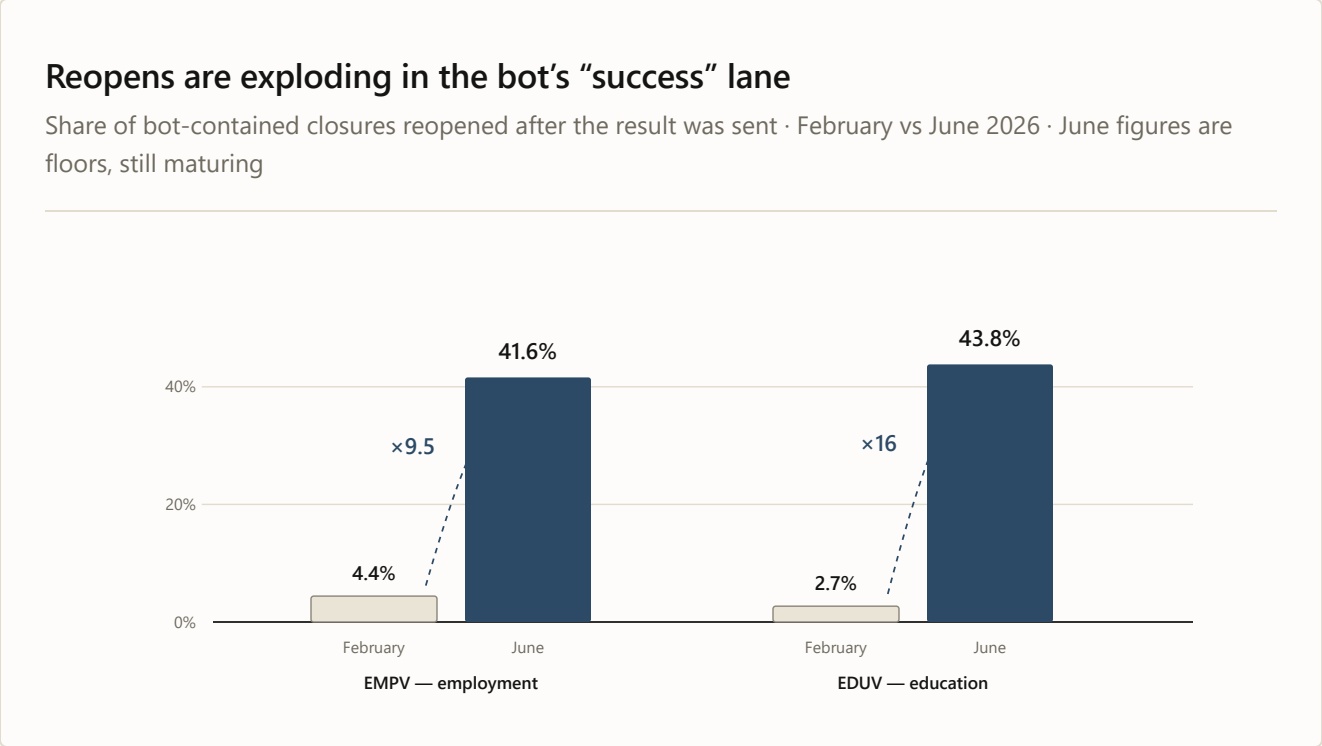


FIGURE 2 Reopen-after-result-sent on bot-contained closures, February vs. June 2026. June postdates are still maturing, so these are floors. The rise is within-lane and concentrated in bot-only closures — which is why the deep-dive's first ask is a closure lock with reopen-reason instrumentation, and why "grow containment" without one would scale a quality problem. The narrow June eForm defect (300 strict cases where the bot emailed *after* a closeout state, 1,119 manual touch rows) is the same missing lock in miniature.

A note is not a handoff

Underneath the volumes sits one mechanism, and the deep-dive names it precisely: *every strict MB touch in the diagnostic packet is note-only*. The bot does work and leaves an annotation — not a state. So the person who picks the case up must reconstruct what happened, whether evidence is complete, and what should happen next. The handoff itself creates work:

The handoff, as-is versus required

The deep-dive's core finding on the left; its required fix — the state contract — on the right

WHAT THE BOT LEAVES TODAY

"Emailed the employer contact. Will follow up."

note only · no outcome state · no next action · no owner · no retry date

WHAT A HANDOFF MUST CARRY (THE STATE CONTRACT)

`outcome_state`	completed · failed · waiting · needs-human · policy-blocked
`evidence_state`	is the case closeable right now?
`selected_contact`	which path it used — no rediscovery
`confidence`	route low-confidence work differently
`next_action`	prevents the "what now?" pickup
`owner`	who holds the case after the touch
`retry_due / cooldown`	stops same-day duplicate outreach
`pickup_reason`	separates valid continuation from rediscovery

FIGURE 3 The note versus the payload. The left side is the deep-dive's finding; the right side is its required fix — and it is, field for field, the shape of the decision engine's evidence-state and logging contracts.

The cost of that missing contract is now measured, not inferred: in the March–May window, humans **re-tried the exact channel the bot had already tried without success** — same method, same case — on roughly 6,700 cases, averaging 2–3 re-tries each (about 16,100 attempt rows), at a median five to six days after the bot's attempt. Scaled to the whole department, that is **roughly 3% of every named human attempt in the window — one touch in every thirty** — spent re-burning a channel the bot had already tried. (Method-level; per-target linkage is exactly what the payload's `selected_contact` field exists to provide.) A handoff that carried "what I tried" would have made each of those a choice instead of a rediscovery.

Every downstream defect in the issue list is the same missing contract seen from a different angle: weak cases enter ungated, channel choices are not preserved, retries lack stop rules, and closures lack a lock. Murphy Brown can execute a touch; it cannot govern the sequence around it.

Murphy Brown automates the motion, not the mission. It can send the email; it cannot run the case.

Why this is the decision engine's story to finish

The decision engine supplies the operating layer the bot is missing: machine-learning models that read every case, a deterministic resolver that knows what is true and what is allowed, a scorer that picks the play, cadence rules that run the sequence to completion, and a log that remembers what was tried. That is not an analogy — the deep-dive's fix list and the engine's component list are the same list, item for item:

Their fix list is our component list

Each measured gap, the control that closes it, and who builds it

MEASURED GAP	REQUIRED CONTROL	OWNER
Closures reopening (~43%)	Closure lock + reopen-reason codes	SNH-AI
Poor-fit pickup (worst decile contains 1.7%)	Score-gated triage eligibility	Joint
Handbacks wait +30.6h	Urgency re-ranking of returns	UBS ops
Unprovable impact	Reversible two-week holdout (~July 15)	SNH-AI
Note-only handoffs (55.5% flow back)	Structured state payload + inbound capture	SNH-AI

FIGURE 4 One list does both jobs: the deep-dive's measured gaps mapped to the decision engine's controls, with the owner of each. The engine-side controls are built and backtested; the bot-side items are the asks, and the field spec for the payload is ready to hand over.

So the proposal is not "turn Murphy Brown off." It is: **Murphy Brown becomes a channel inside the decision engine.** The resolver decides *when* the bot may touch a case; the difficulty scores decide *whether it should* (fax-primary and thin-contact cases skip it — the deep-dive's own modeling says gating to the top ~60–70% keeps 83–91% of the bot's wins while halving the handback lane); terminal states and the inbound-first rule *lock what's closed*; the priority queue re-ranks what comes back; and the flight recorder finally makes the bot's value a measured number instead of a debate.

What I'm careful not to claim

- **Association, not causation.** The bot is *associated with* a large downstream queue. The matched comparison does not prove it makes comparable cases worse on effort or time — the proven finding is the residual queue, the weak fast-closeout, and the missing handoff state.

- **No savings claims.** The handback lane's labor premium is small (≈$3/search). The money case is turnaround time and closure quality — 100,000+ EDUV wait-hours in the window, and reopen churn now measured at 0.69–3.4 reopens per automated closure (the leak-back band) but still unpriced in dollars. Attempt rows are not labor minutes, FTEs, or ROI.
- **No avoidable-percentage.** No share of the bot's downstream work is labeled "avoidable" — the record doesn't yet carry enough per-case evidence to make that judgment stick, so no avoidability number is quoted. The causal number comes from a two-week 50/50 holdout — one configuration rule on the bot's side, fully reversible — with a before-vs-after comparison against untouched cases running ~July 15 regardless.
- **Modeled is labeled modeled.** The missed-pickup mechanism (all-or-nothing API routing; about 50,000 exposed searches, ~7.2–8.6k modeled manual searches/month) still needs effective-dated configuration history to be proof-grade; the 44.9% neither-automation figure is measured per ticket where automation is live.
- **And one test the bot passed.** I probed whether MB fires before evidence exists: 0.0% of its touches happen while a vendor clock is open, and only 0.2–1.7% land before a vendor return. The bot's timing discipline is not the problem — its handoffs, closures, and gating are.

What the bot becomes

The same bot, two operating realities

What changes for Murphy Brown when the decision engine runs the case around it

MURPHY BROWN TODAY	MURPHY BROWN INSIDE THE ENGINE
Picks up whatever has a pre-laid contact path	The ML models pick the cases the bot is suited to win
Executes one scripted touch	Deterministic rules confirm the touch is allowed first
Leaves a note behind	Writes structured state and a next action
Returns uncertain work to the back of the queue	Handbacks re-enter ranked by risk and urgency
Cannot learn from the outcome	Every outcome becomes evidence for the next play

FIGURE 5 The same five behaviors, before and after governance. The left column is the measured present; the right column is the decision engine's existing components applied to the bot.

Inside the engine, the model fleet chooses the fit, rules guard the touch, and a person approves. Murphy Brown executes. As the evidence grows, AI can repair dead contact paths and an SLM can draft the outreach — but the bot's durable role is simpler: it becomes the engine's fastest pair of hands, and every outcome improves the next case. ◆

The series. This field report is the companion piece. The system it argues for: the executive briefing · the engineering deep dive · the design philosophy · the illustrated walkthrough.

Sources of record in the ops deep-dive workspace: `MB_MURPHY_BROWN_ALL_FINDINGS_CONSOLIDATED_20260630.md` and `outputs/exec_packet_20260702/MB_ONE_PAGER_20260702.md` (which supersedes earlier summaries; retired numbers per its `RETIRED_NUMBERS.md` are not cited here). All analysis read-only; windows as stated per figure; no client names, personnel names, or case identifiers appear in this report.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

Meet **case 4127**: an employment verification, opened on a Monday. There is a work email and a phone number on file, no fax. Somewhere, an HR inbox is about to be asked to confirm a job history. Over the next nine days this case will be emailed twice, called once, and finally verified — and at every step, the system has to answer the same two questions: *how is this case doing?* and *what, if anything, should we do next?*

Case 4127 will not move through one model. It moves through a sequence of small decisions, each allowed to answer only one question — five components taking turns. Here is the cast, and the color code every diagram in this post uses:

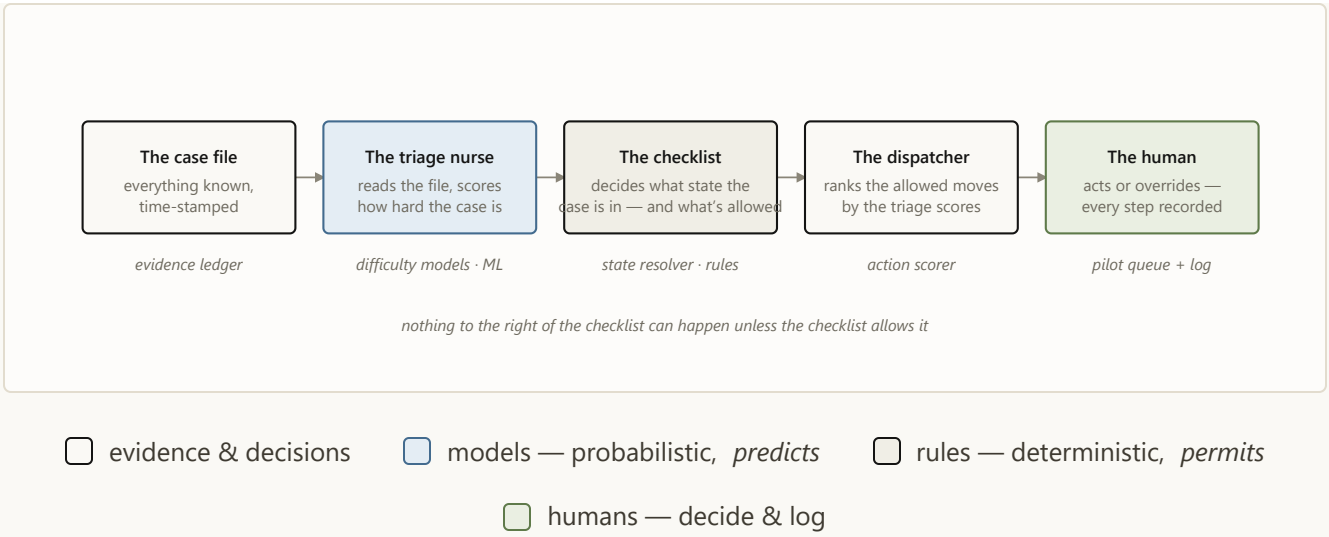


FIGURE 1 The cast, in the order they speak. The color code holds for every diagram below: blue predicts, oat permits, moss decides.

1 The curtain: what the system is allowed to know

The single most important idea in the architecture is not a model — it's a rule about time. The system never looks at a case as one blob of history. It looks at the case *as of a specific morning*, and it may only use facts from **before** that morning. One case therefore becomes many training rows — one per day it stayed open — and our twelve months of history becomes 2.75 million of these `(case, day)` snapshots.

Drag the day below and watch what case 4127 looks like from inside the system. Everything right of the curtain hasn't happened yet — as far as the machine knows, it never will.

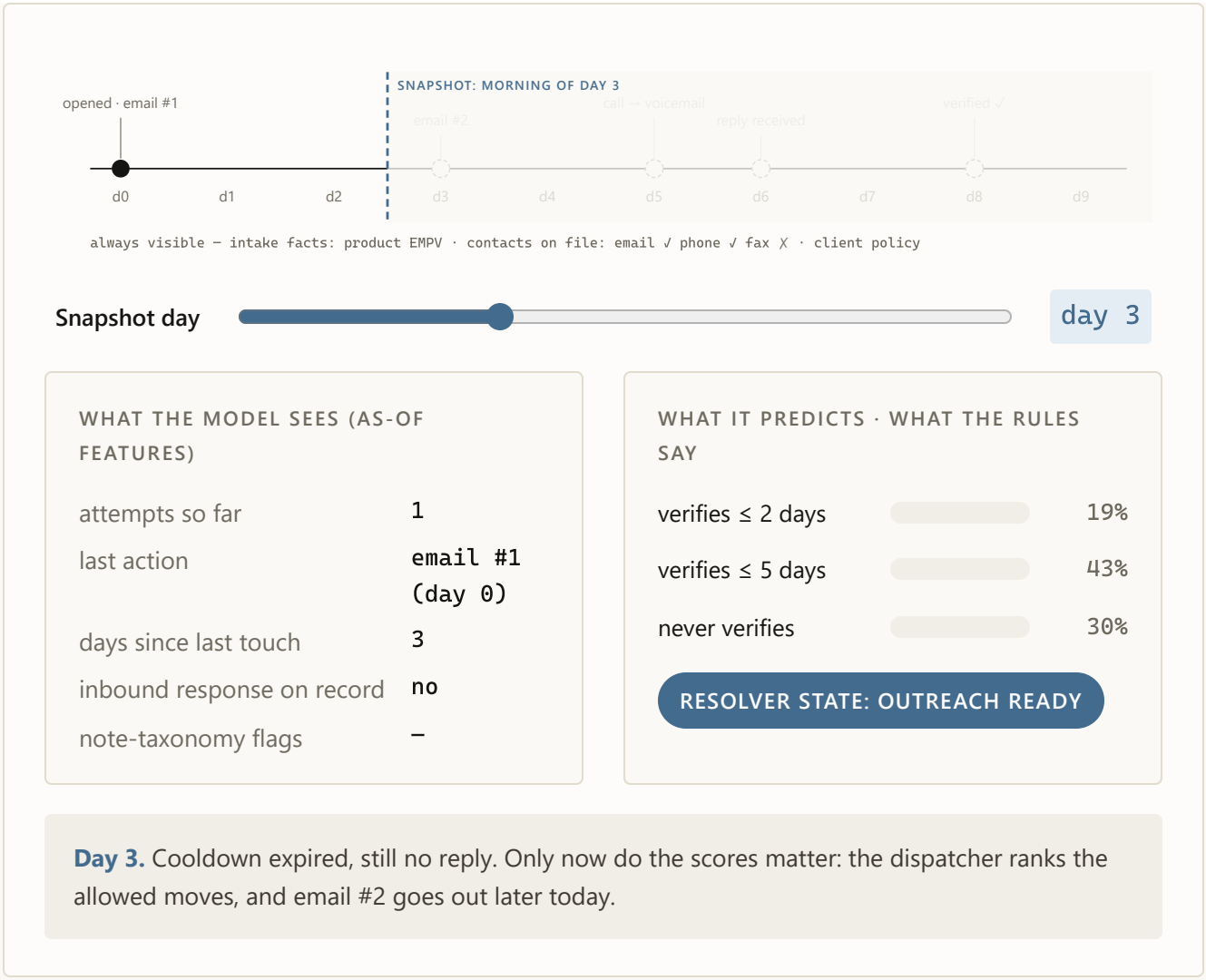


FIGURE 2 · EVIDENCE SHOWS The as-of curtain, interactively. Probabilities are illustrative. The curtain is enforced, not assumed: an automated audit re-checks it on every new training matrix before any model trains.

2 Triage: three questions about the same case

For case 4127, Monday morning — the day it arrives — is the sharpest read it will ever get, before the first attempt changes the evidence. Each morning after, three forecasting specialists from the wider model fleet re-score the visible half of the file: *will this verify within 2 days? within 5? will it ever?* Calibrated means the numbers behave like frequencies — across all cases

scored “30% never-verify,” about 30% actually never verify. That property is what makes the scores safe to build queues and thresholds on.

One case’s scores are mildly interesting. The population view is where triage earns its keep: sorted by predicted never-verify risk, the easiest tenth of cases fails 3% of the time and the hardest tenth 48%. Same process, same operators — a 16× spread in what was ever achievable. That spread powers the queue (work the winnable middle first) and a fairer scoreboard (measure misses against what was possible, not against a blended average).

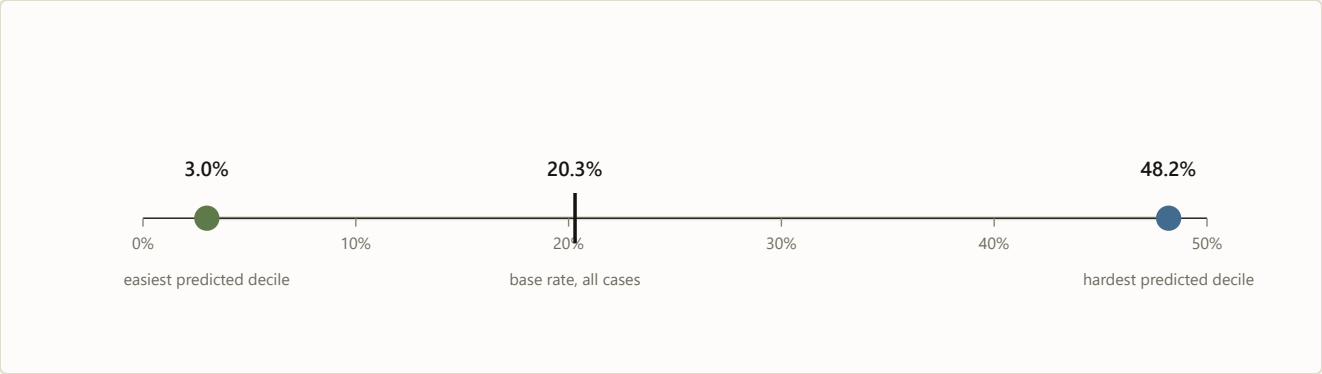


FIGURE 3 · ML PREDICTS Real numbers (May 2026 holdout): observed never-verify rate by predicted-risk decile. Triage doesn’t change any single case; it changes which cases get the attention.

3 The checklist: what state is this case actually in?

By day 3, the live question about case 4127 is no longer how hard it is — it is what is true this morning: did anyone reply? is a cooldown running? is another touch allowed? Before anyone acts on a score, a deterministic resolver asks a fixed sequence of questions any good supervisor would ask — and the first “yes” decides the case’s state for the day. The real resolver distinguishes thirteen states; the intuition fits in seven questions:

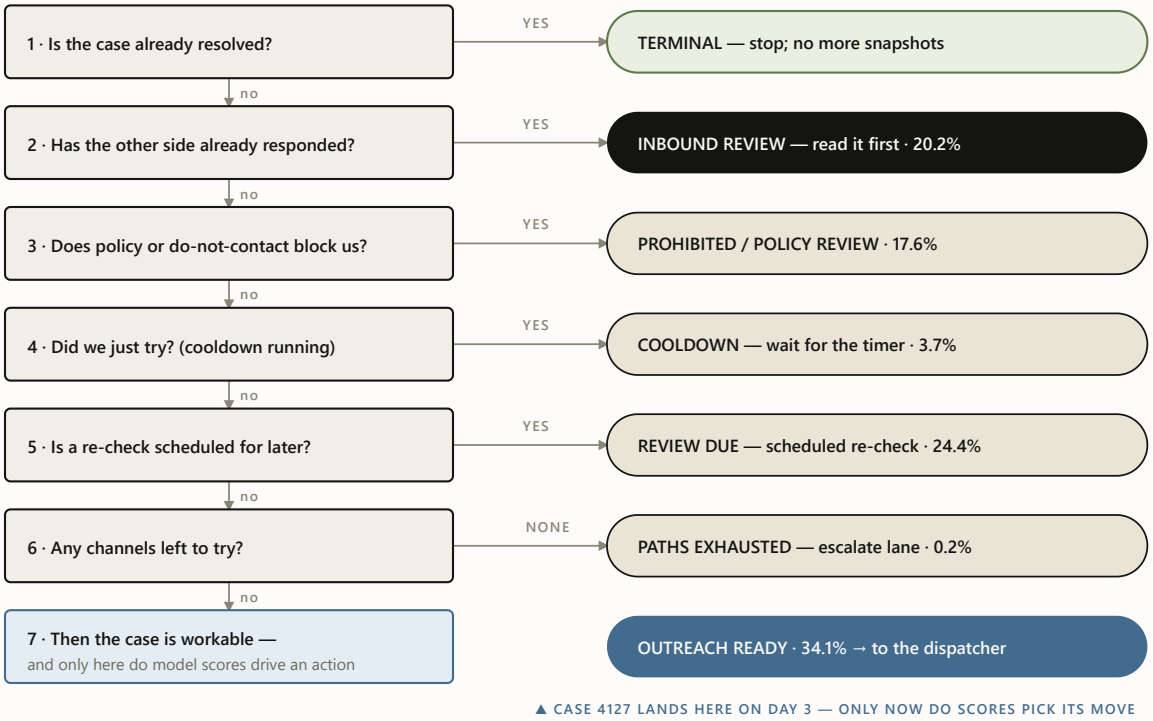


FIGURE 4 · RULES NAME The resolver as a supervisor's checklist (a seven-question simplification of the real thirteen states; percentages are each state's share of all 2.75M case-days). Notice question 2: a fifth of all case-days already have a response on record. Answering it deterministically is what caught the ~1-in-7 cases the old flow would have re-contacted. Across all 2.75M case-days: zero do-not-contact violations, zero policy violations — by construction, since scores are never consulted until row 7.

4 The bouncer and the dispatcher: six cards, one recommendation

Suppose it's day 3 and case 4127 landed on "outreach ready." The system now deals six possible moves — the same six for every case, 16.5 million candidate cards in the full run. Eligibility rules physically remove cards before any ranking happens; the model's opinion of a blocked card is irrelevant, because the card is no longer on the table.

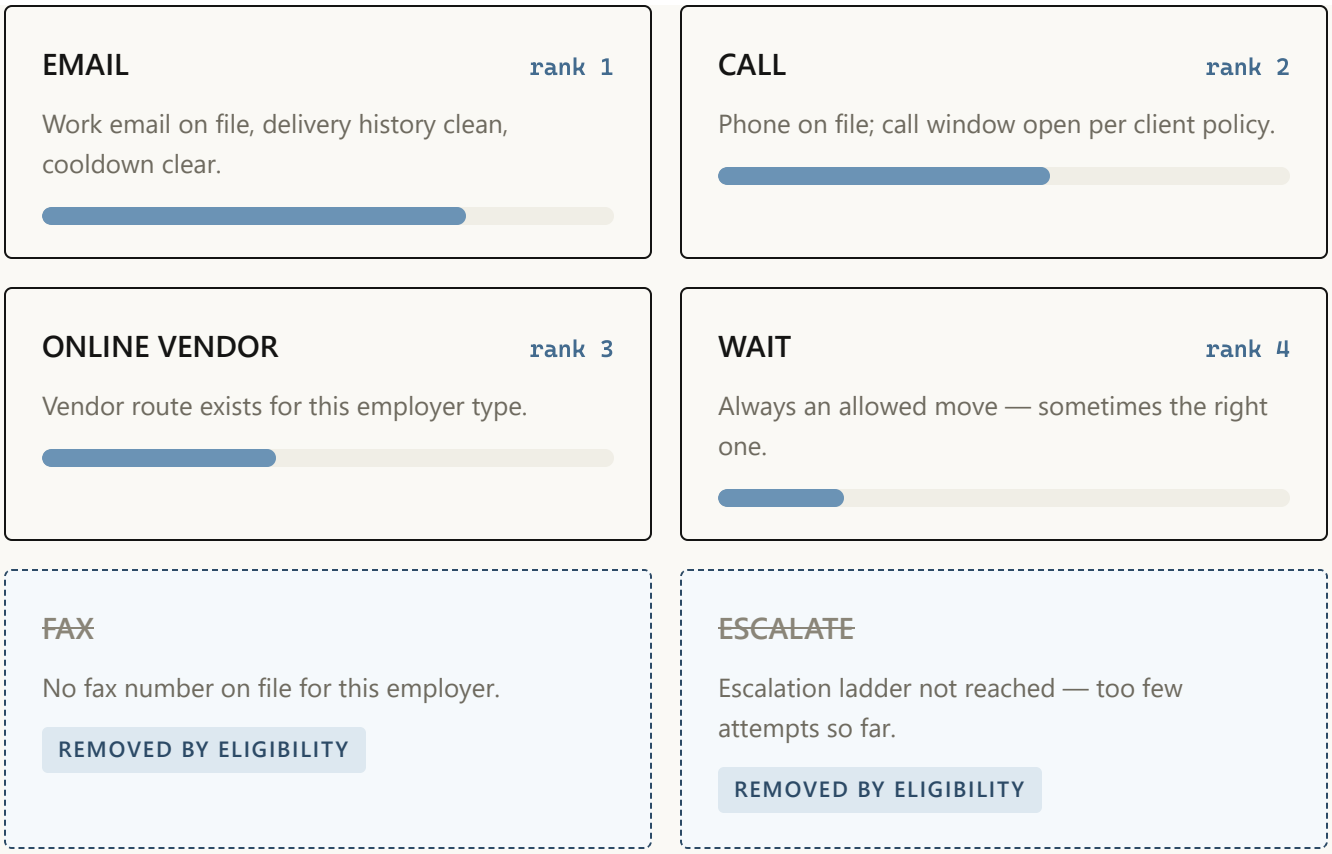


FIGURE 5 · RULES REMOVE, ML RANKS Case 4127's hand on day 3. Priority bars are illustrative; ranking reflects what historically moved similar cases under current policy — an observational signal, deliberately not a claim that any channel *causes* better outcomes. Blocked cards never reach the ranker: eligibility is a filter, not a penalty.

The dispatcher's output for the day is exactly one line: *recommended next move: EMAIL, rank 1 of 4 allowed*. In the full backtest, every single case-group ended with an available move — zero "rank-1 but blocked" contradictions, zero empty hands.

5 The flight recorder: a person decides, everything is written down

The recommendation lands in a queue in front of a person, who can take it or override it — and the override is as valuable a signal as the action. Five event types get logged, and together they form the case's complete decision story:

day 3 · 09:12	RECOMMENDED EMAIL — rank 1 of 4 allowed moves, shown to operator
day 3 · 09:14	ACTION EMAIL sent, no override · template family `follow_up` v3
day 4 · 07:40	DELIVERY delivered — no bounce
day 6 · 15:03	REPLY inbound response recorded → resolver flips to INBOUND REVIEW
day 8 · 11:26	OUTCOME verified — outcome joined back to every prior event

FIGURE 6 · PERSON DECIDES, LOG REMEMBERS Case 4127's flight record (fictional timestamps). This chain — recommendation, action or override, template version, delivery and reply, outcome — is the price of ever saying the word "because." Until it's collected at scale, the system reports correlations as observations and nothing more.

6 The heartbeat: how the machine stays honest month over month

Case 4127 verified on day 8; its complete decision story joins the next monthly evaluation cycle. New evidence rebuilds the same as-of snapshots, challengers train behind the same leakage audit, and the promotion gate records whether each specialist advances or stays put.

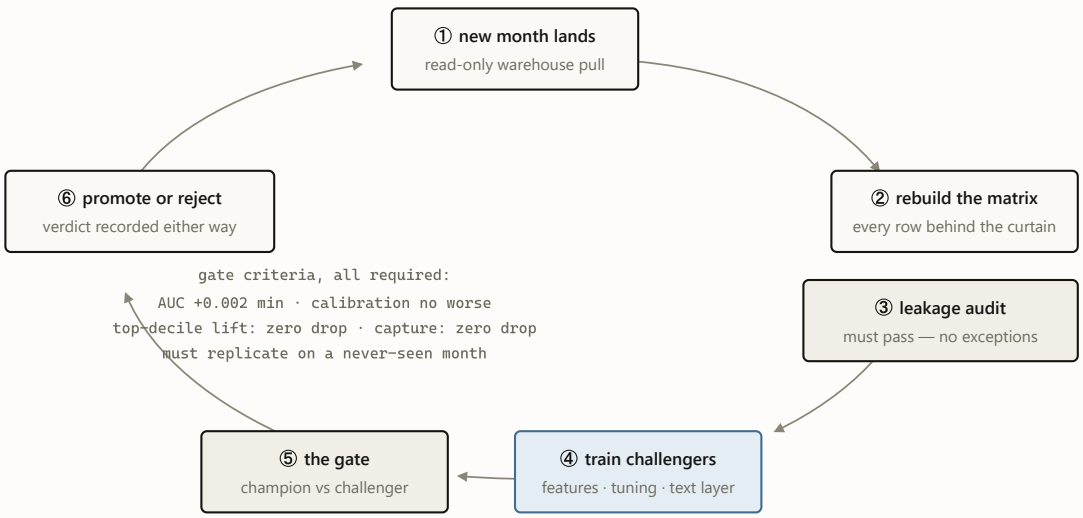


FIGURE 7 · OUTCOMES TEACH The monthly heartbeat. July 2026's cycle promoted text-aware champions for both success targets and rejected the same idea for the never-verify target — its third rejection this year, each one recorded in the champion manifest. A gate that never says no is a rubber stamp.

WHY TWO LANES AND NOT ONE BIG MODEL?

Because prediction and permission fail differently. A poor ranking wastes effort; a poor permission creates an incident. The learned lane handles the recoverable error, and the deterministic lane owns the boundary that cannot move — a single end-to-end model would blend the two and price everything at the expensive one.

The whole machine today — and what case 4127 looks like next

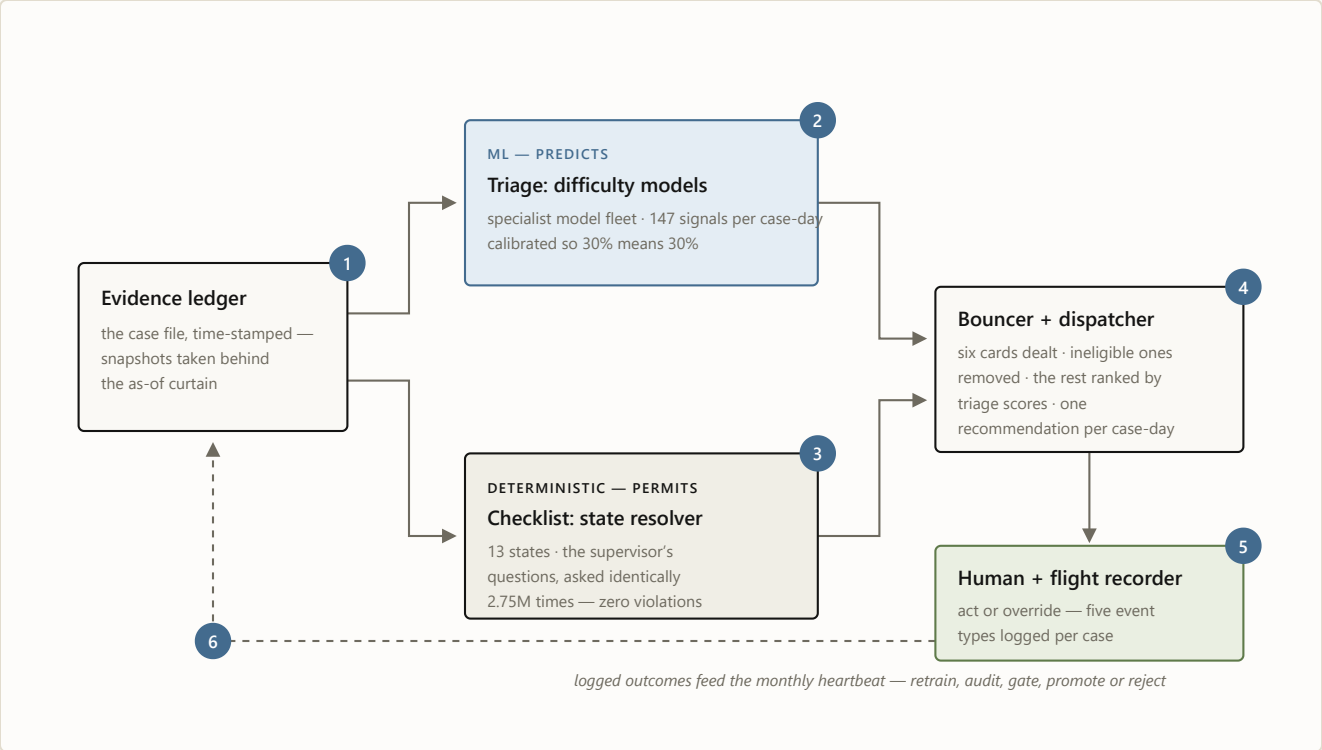


FIGURE 8 The assembled machine, with case 4127's six stops numbered in order: **1** the file is snapshotted behind the curtain · **2** triage scores it · **3** the checklist names its state · **4** the bouncer deals and the dispatcher ranks · **5** a person decides and the recorder writes it down · **6** the logs feed the monthly heartbeat. Blue predicts; oat permits; moss decides.

Now run the same case forward a stage. Today, the engine recognizes that case 4127 is ready for outreach, removes the blocked moves, and recommends email; a person reviews and sends it. In the next stage, the engine recommends the *complete play*: confirm the contact path is still good, email this morning with the employment-confirmation message family, and schedule a call only if no reply lands inside the measured response window. If the path on file looks dead, an AI contact researcher hunts a fresh, verified one before another attempt is spent. A small language model drafts the email; a person approves or edits it, and the system records why. And if that complete play keeps

winning across comparable cases, that one narrow lane can move to controlled auto-send on its own logged record.

The machine does not change; what it can safely recommend grows. Today it recommends the next move. Tomorrow it learns the complete play from every finished case. ◆

The series: Building a decision engine for verification casework (the engineering deep dive) · Earned autonomy (the design philosophy) · this illustrated guide · plus the executive briefing and the Murphy Brown field report.

Case 4127 is fictional and its probabilities, ranks, and timestamps are illustrative. Real numbers cited: 2,751,900 case-day snapshots; 16.5M candidate actions; state-mix percentages from the 2026-06-19 resolver run; the 3.0% / 48.2% decile spread from the May 2026 holdout; zero DNC and policy violations across the full backtest. Correlations are observational — no channel or wording is claimed to cause outcomes.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · July 2026

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

Every verification case ends one of two ways: verified, or *unable to verify*. In the twelve months ending May 2026, UBS worked nearly a million verification searches across four products — employment, education, reference, and license-style checks (EMPV, EDUV, REFV, PLV). Just under half a million of them received at least one attempt, and one in five of those — roughly 100,000 — ended unable to verify. That UTV rate is the number the business wants down, and it is a frustrating one to manage, because it blends two very

different populations: cases that were never going to verify no matter what anyone did, and cases that were winnable but stalled.

The system described here exists to separate those populations and act on the difference. For every open case on every day it answers two questions — *how hard is this case?* and *what is the next sensible move?* — and it answers them with a strict division of labor: machine-learned models predict, deterministic rules permit. This post explains how the pieces fit, what the evaluation discipline looks like, and the places the discipline said no. One expectation to set early: the largest immediately bankable finding in here did not come from a better model. It came from writing down, in rules, what "already responded" means.

Control changes hands five times

The architecture is five layers, and the arrows matter more than the boxes.

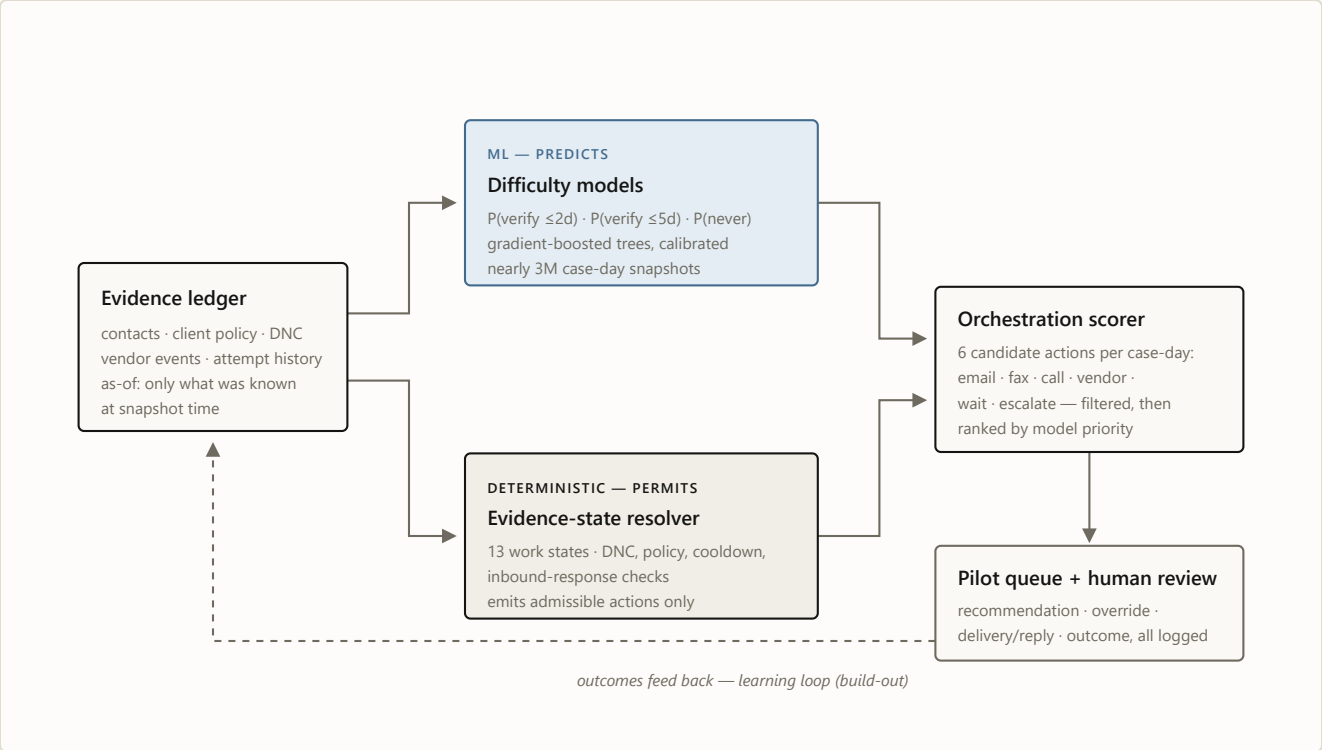


FIGURE 1 The decision path. Both halves read the same as-of evidence; only the resolver decides what is allowed. The dashed loop — learning from logged outcomes — is the build-out, not the current system.

The load-bearing rule sits in the middle: **the resolver alone defines which actions exist. Models rank only the actions that survive.** A blocked action is removed, not penalized, so no level of model confidence can restore it.

Three questions, one matrix

Three forecast specialists answer the questions that drive daily triage: will this verify within two days, within five days, or ever? The wider fleet adds intake, aging-case, and early-warning reads — each specialist with its own target and validation record, all reading the same morning snapshot: one per open case per day, nearly three million over the twelve-month window, each restricted to what was knowable that morning. The models themselves are gradient-boosted decision trees (the HistGradientBoosting family): an ensemble

method that builds hundreds of small decision trees, each one trained to correct the mistakes of the trees before it. I chose the family deliberately over deep-learning alternatives: on structured operational data it is still the benchmark to beat, it handles counts, categories, and missing values natively, it retrains in minutes, and its decisions can be interrogated feature by feature. On top of each model sits a calibration layer that turns raw scores into honest probabilities — when the system says 30%, it happens about 30% of the time — because the consumers of these scores are queues and thresholds, and a queue built on miscalibrated probabilities is a queue that lies about its own workload.

Table 1. Champion models (v3, promoted 2026-07-02). May 2026 is the holdout gate month — roughly 300,000 nev 2026 is a further out-of-time month scored with frozen bundles.

TARGET	CHAMPION VARIANT	BASE RATE	ROC-AUC, MAY	ROC-AUC, JUNE O
Never verifies (final UTV) the headline target	baseline, bounded features kept — 3 challengers rejected	0.201	0.706	champion held; lift 2.61× vs 2.4
Verifies within 5 days	note-aware, identity-safe promoted	0.477	0.717	0.7
Verifies within 2 days	note-aware promoted	0.264	0.747	0.7

In plain terms: the never-verify specialist places the eventual failure ahead of the eventual success **71% of the time**, versus 50% for chance (that is what

ROC-AUC 0.706 means; every score here reads the same way). The two-day and five-day specialists sort at 75% and 72%.

At intake — before any work has happened — the identity-safe day-0 variant reaches **≈81%** across three forward folds. That is not decay in reverse: fresh cases differ sharply, while cases still open a week later increasingly resemble one another, so ranking the survivors is intrinsically harder. What the fleet buys operationally is the spread: the tenth of cases it ranks riskiest actually fails **48% of the time against a department-wide base rate of 20%** — 2.4× the true never-verifies packed into the first list an operator works:

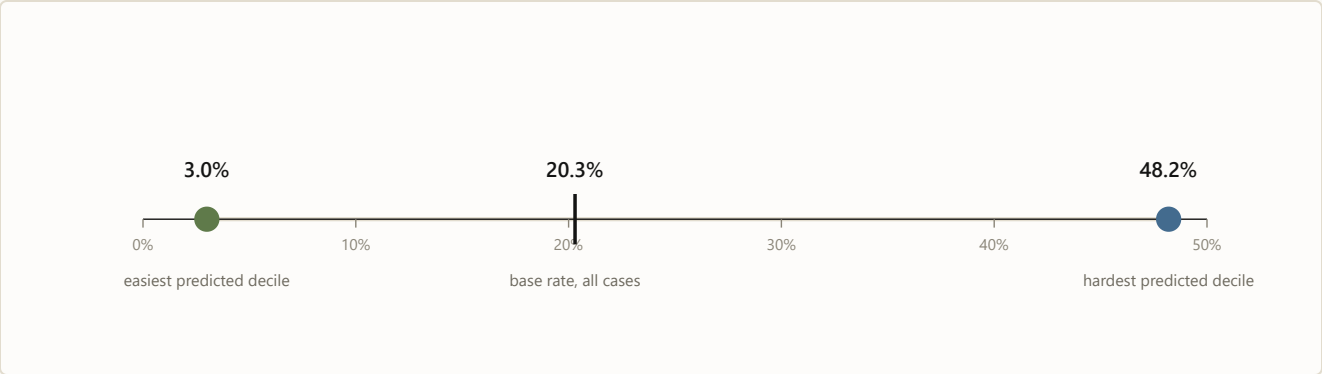


FIGURE 2 Observed unable-to-verify rate by predicted-risk decile, May 2026 holdout. The model separates cases that fail 3% of the time from cases that fail 48% of the time. That spread is what makes a *fair* UTV metric possible: misses can finally be measured against what was actually winnable.

That 16× spread is also the business story. Today's UTV rate silently blames operators for cases that were structurally dead on arrival. Score-conditioned targets flip that: the easy decile is expected to verify almost always, the hard decile almost never, and management attention goes to the winnable middle.

The three champions are also just the tip of the fleet — each specialist with its own validation story:

Table 2. The model fleet. All are gradient-boosted trees, calibrated so their probabilities read true (a 30% means 30%), trained under the same as-of matrix discipline; every model passes the leakage audit, and champions additionally clear the promotion gate.

MODEL	PREDICTS	ROC-AUC	VALIDATION	STATUS
Never-verify champion	will the case ever verify	0.706	May holdout + June OOT	champion — bounded
Verify within 2 days	resolves ≤2d of snapshot	0.747	May holdout + June OOT	champion — text features
Verify within 5 days	resolves ≤5d of snapshot	0.717	May holdout + June OOT	champion — text, identity-safe
Day-0 intake read	never-verify, at arrival	~0.81	walk-forward, 3 forward months	identity-safe variant
Day-1/2 re-score	re-reads aging cases	0.73 / 0.70	holdout	lifts aging EMPV read 0.62 → 0.71
Early warning	will strand by day 20	0.776	single-month holdout	preliminary

A high score still cannot break a rule

The models answer “what is likely?” The resolver answers “what is true, what is blocked, and what may happen next?” The questions stay separate because they fail differently: a bad ranking wastes effort; a bad permission creates an incident. The deterministic half is an evidence-state resolver that classifies every case-day into one of thirteen work states — terminal, already-responded, policy-blocked, awaiting response, cooldown, review-due, research-needed, digital-paths-exhausted, standard-paths-exhausted, outreach-ready, and so on — and derives the set of admissible actions from the state, never from a score. Here is where the 2.75M snapshots actually sit:

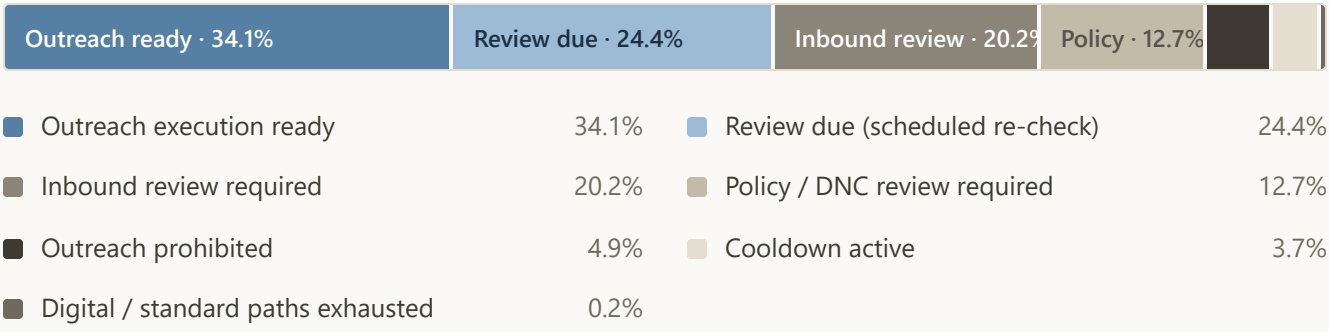


FIGURE 3 Resolver state mix across nearly three million snapshots. The interesting segment is the third one: cases where a vendor response is already on record but the legacy flow would have kept contacting.

I expected the next win here to come from a better model. It came from a rule — and it was hiding in that third segment. On roughly **400,000 case-days — about one in seven** — a vendor-channel response had already been received, yet the process as practiced would have actively re-contacted the party. The resolver routes every one of them to review-first instead. No model was needed for this; it fell out of writing down, deterministically, what "already responded" means. (The signal is a vendor receipt, not parsed reply content, and it doesn't exist for every product — the honest version of this number lives in the repo docs alongside its caveats.)

The same rule also shows why the safety claim holds under pressure. Suppose a model strongly favors another call on one of those cases. The resolver sees that a reply is already on record, so call is removed before the final ranking — the model's confidence cannot restore it, and the person never even sees it as a recommendation. That is why the replay produced zero policy violations: safety does not depend on the model remembering every rule. The model never receives the authority to break one.

Downstream of the resolver, the scorer builds six candidate actions per case-day — email, fax, call, online vendor, wait, escalate — 16.5 million candidate

actions in the full run, filters them through eligibility, and ranks the survivors by the calibrated scores. A seven-stage pipeline chains candidates → eligibility → scoring → call-escalation ladder → wait policy → workflow router → pilot queue; every stage checks its own output and halts the run on any failure, and a strict contract validator signs off at the end. The full run produced zero blocked rank-1 actions, zero no-action groups, and a queue where every group has an available move.

One case-day, from evidence to action

Here is the whole handoff on a single day, using the illustrated guide's fictional case 4127. On day three, the case enters as a morning snapshot. The machine-learning models estimate its likely path. The resolver names it outreach-ready. The rules remove fax, because no fax number exists, and remove escalation, because the case hasn't reached that rung. The scorer ranks the four surviving moves and places email first. A person approves the recommendation, and the log records the whole chain, recommendation to final outcome.

The models never see six equally available actions. The operating system first decides which actions are real.

No peeking, by construction

The easiest way for a model like this to look brilliant is to let information from after the outcome leak into its features — post-outcome status fields, completion timestamps, identity proxies. Every feature here is built under strict "as-of" discipline instead: 117 columns, each computed with strictly-

before semantics. An automated leakage audit re-checks that boundary on every new training matrix — and again at every monthly retrain. The scores in this post are conservative by design, measured the hard, honest way.

The system rejected three of my improvements

Model improvements are promoted per target — there is no single "champion model," there are three — and promotion is mechanical. A challenger must clear every criterion against the sitting incumbent; a single checked-in champion config is the source of truth, and rejections are machine-recorded in the champion manifest.

ROC-AUC	at least +0.002 over the incumbent, with bootstrap confidence intervals excluding zero
Brier score	no worse than the incumbent — calibration is not negotiable
Top-decile lift	zero drop tolerated — the top of the queue is what operators actually work
Capture at top 10%	zero drop tolerated
Slice gate	per-product and per-day slices, zero-tolerance with a reviewed, unit-tested allowlist for false positives
Out-of-time replication	the win must survive a never-seen month scored with frozen bundles

The gate earns its keep on what it rejects. Three times this year I tried to improve the never-verify model — a settings-tuning pass, a guarded version of

the same sweep, and this cycle's note-reading variant — and the gate turned all three away **for the same reason**: each won on average, and lost where operators actually work — the top of the ranked worklist, the first page, where accuracy is worth the most. One rejection is bad luck; three identical ones is a message. For this target, average accuracy and top-of-list sharpness genuinely pull against each other, so the next attempt changes the training data itself — June's outcomes finish maturing in the July pull — rather than pushing a fourth model variant.

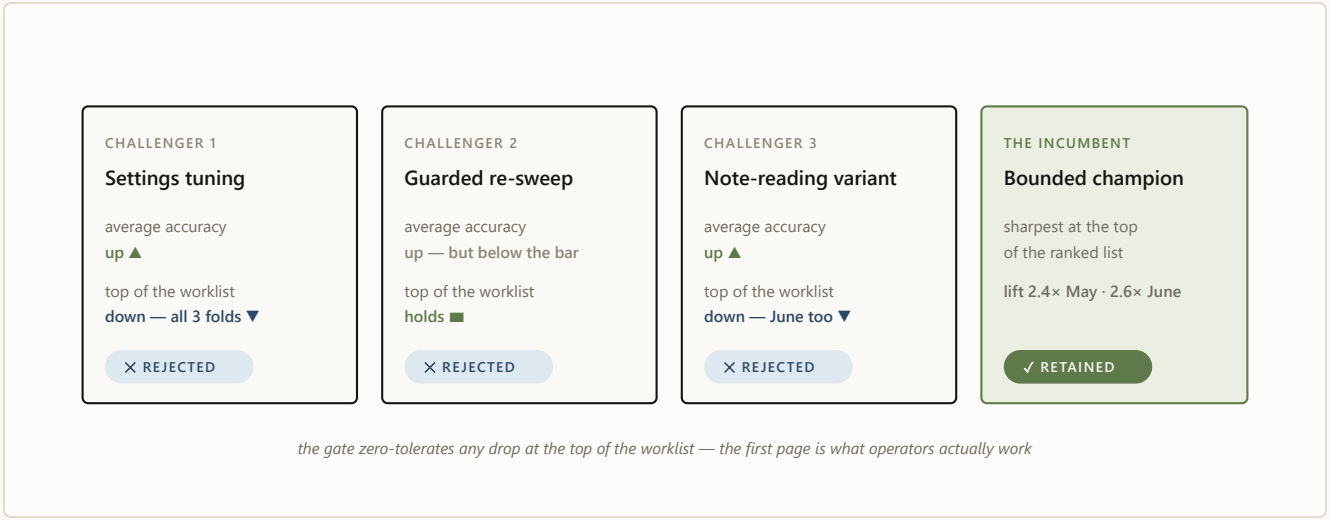


FIGURE 4 The gate at work on the never-verify target: three challengers, three different levers, one identical failure mode. Each verdict is machine-recorded — the rejections are as documented as the promotions.

Reading the notes without keeping them

The biggest accuracy gain this cycle came from the data source I had been most careful to avoid: the free-text notes operators and vendors leave on case history ("left a voicemail"; "reached HR, call back Tuesday"). Notes are where the richest signal lives — and where the risk lives twice over: outcome language ("verified via…") leaks answers into training, and raw text carries the privacy exposure, so persisting it into training artifacts was never on the table.

The design that threads the needle: **a deterministic reading layer classifies each note the moment it streams past — then throws the text away**. Every note is sorted into eleven kinds of operational signal (contact made, voicemail left, callback promised, redirected to a third party, …), and only the signal survives: which kinds of events a case has seen, how recently, how often. Deterministic matters here — the same note always produces the same labels, so the privacy boundary has no black box in it and the whole layer can be audited and replayed. That turns 17.9 million unreadable notes into 30 new machine-usable signals per case-day for the learned models, computed under the same no-peeking time discipline as everything else; about 28% of case-days carry at least one.

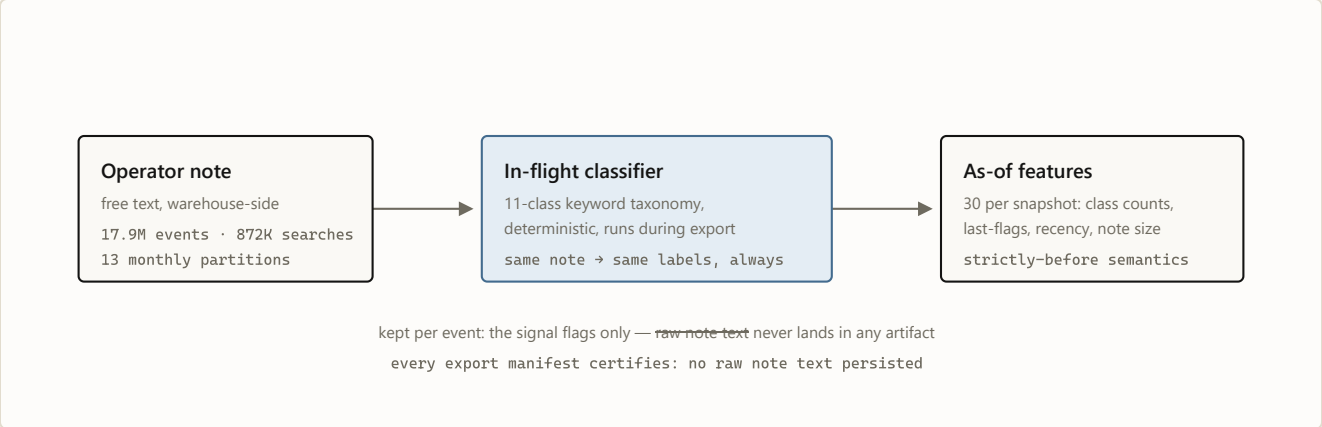


FIGURE 5 The inbound text layer. Signal crosses the boundary; text does not.

The same new evidence improved the two success forecasts but weakened the top of the never-verify worklist. Two challengers advanced; the third was rejected.

One operational note survives from that build: a routine row-count check against the export manifests caught a corrupted export *before anything consumed it*. Cheap invariant checks between pipeline stages are the highest-ROI code in the system.

What the evidence still will not let me say

The layers ahead are ambitious, which makes the boundary between built, ready, and not-yet-proved more important — not less. The repo maintains an explicit disallowed-claims list, and it does more for the project's credibility than any AUC. The current lines:

- **No causal claims about channel, wording, or templates.** Historical patterns like "email-first cases verified more often than call-first cases" are observational — selection effects all the way down. Causal language stays banned until the pilot logs the full chain for each case: recommendation shown, action taken or override reason, template version, delivery/reply, final outcome. The logging schema for exactly those five event types is built and validated; the evidence is not yet collected.
- **Not "identity-free," only "direct identifiers stripped."** Action-facing models exclude direct agent and team IDs behind a drift gate, but behavioral agent attributes remain. Identity-bearing champions stay `provisional` for action-facing use, and the resolver applies no-direct-identity fallbacks automatically until that gate approves.
- **Nothing auto-sends today.** Contact research, message drafting, and controlled auto-send remain evidence-gated future capabilities; each begins with human review and advances only on its own logged outcomes. The router is shadow-review metadata, not an autonomous policy — and I won't train a "learned policy" on historical action logs and pretend the logs were experiments.
- **No rework-savings claims — the QA loop is measured, not yet moved.** A reproducible July baseline timed the quality-review loop (education + employment, Jan–Jun 2026): a returned check runs a median

11.2 days from first QA return to approval, against 5.0 days for a typical check's entire first pass — and 85.6% of coded return reasons are process-and-judgment classes, the ones the resolver, eligibility rules and lanes exist to prevent. "Exist to prevent" is a design statement, not an effect size: the effect claim waits for the pilot, and hands-on handle time waits for work-item state logs that nothing emits today.

What the live record teaches next

The engine today solves three foundational problems: it reads the case, decides what is allowed, and recommends the next move. Each layer that follows expands what a "move" contains — and each becomes possible because the one below it already exists. The resolver supplies the state; the rules supply the boundaries; the pilot supplies the evidence. Concretely: the recommendation is logged; the person accepts or overrides it; the chosen channel, timing, message, delivery, reply, and final outcome are recorded. That complete chain — not the final outcome alone — is what teaches the system which play worked.

What gets built on it, in order of readiness: the **July cycle** matures June's never-verify labels — the most promising lever for the one target that keeps rejecting challengers — while the remaining pre-integration checks either approve the identity-bearing champions for action-facing use or keep the fallbacks in place. Then the **play engine** learns the complete play — channel, timing, message family, template — from pilot data that records which options were offered, not just which was taken. The **contact researcher** repairs the inputs themselves — fresh, verified emails, fax numbers, and phone numbers entering as candidates for the rules to admit — so unworkable cases turn

workable before attempts are wasted. And the **SLM drafts the outreach** last because it is graded last: only the pilot's causal instrumentation can tell a well-worded email from a lucky one.

Automation arrives at the end, and is earned: every lane starts with human review, and a narrow lane moves to auto-send only on its own logged record — one lane at a time, never from a roadmap date.

The system today is, on purpose, the boring version: calibrated probabilities, deterministic guardrails, mechanical promotion gates, and a written list of things I refuse to say. Every part of it is replaceable — the models get retrained, the rules will evolve, the contact researcher and the language model still have to earn their place. What persists is the handoff: evidence enters, machine learning predicts, rules permit, a person decides, and the outcome becomes the next case's evidence. The models provide the intelligence; the operating system turns it into work the operation can trust. ◆

The series. This is the engineering deep dive. Also: Earned autonomy (the design philosophy) · the illustrated walkthrough (one case through the machine) · the executive briefing · the Murphy Brown field report.

SNH CAPITAL PARTNERS *A Wright Original* · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · July 2026

A Wright Original · Built by **SNH Strategic Initiatives**

Operational casework looks made for end-to-end AI: large caseloads, daily decisions, hard compliance constraints, delayed and noisy outcomes, and human operators accountable for the result. It is also where naive autonomy fails fastest. Policy is not a weighted preference, historical actions are not experiments, and a plausible recommendation is not evidence that an action should be allowed. Verification work is our instance; collections, claims, and outreach operations all share the shape.

Our current verification bot is the live companion example: fast at execution, weak wherever the work requires state, sequence, or judgment — the field

report measures it.

So I built the opposite of an end-to-end agent — because in this class of work, the fastest route to useful autonomy begins by refusing it. The interesting part isn't the system; it's the discipline. Four rules, in increasing order of how much they cost us.

1. Split the decision

Every "what should we do next?" question decomposes into two questions with different failure tolerances. *What is permitted?* has a tolerance of zero: getting it wrong is an incident, and the answer must be explainable after the fact, case by case. *What is preferred?* has a tolerance of "be usefully better than the status quo": getting it wrong wastes effort. The pattern's first rule is to never let one component answer both.

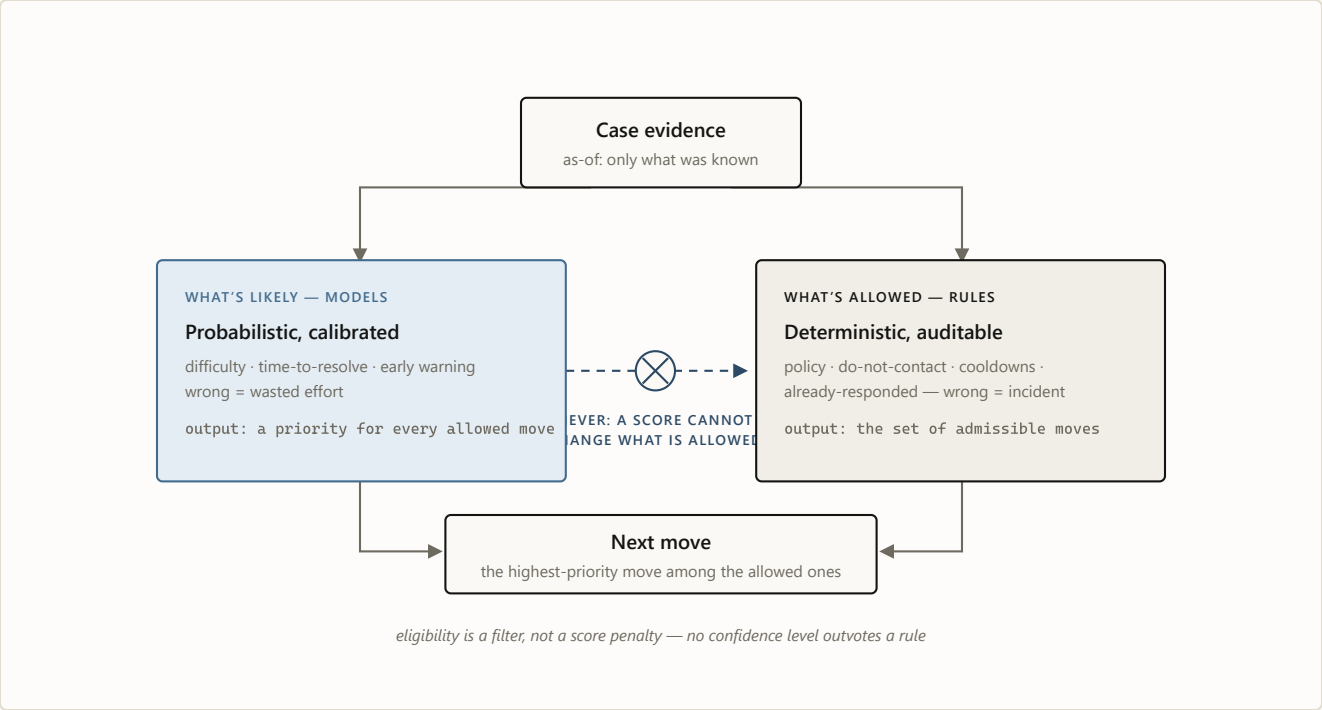


FIGURE 1 The split. Both lanes read the same evidence, but only the deterministic lane can say no. In this system the rules lane resolved a full year of case-days into thirteen work states with zero do-not-contact and zero hard-policy violations — an unremarkable number, which is the point: the models never had the opportunity to create one.

The split also produced the cheapest large win in the project, and it came from the *rules* lane: recognizing, deterministically, that a reply was already on record before another contact could occur. No model, no training run — just a definition, applied consistently at scale.

2. Forecast first, act later

The second rule is about sequencing: the first machine-learned product should be a *forecast*, not an action. Forecasts create value before they receive authority — for triage, for early warning, and for what turned out to matter most to the business: **fixing the metric itself**. The first product was not an automated action but a fairer view of difficulty.

An aggregate failure rate blames operators for cases that were structurally dead on arrival. A difficulty model unblinds that: the easiest predicted decile fails 3% of the time, the hardest 48%. Once you can condition on difficulty, "how are we doing?" becomes "how are we doing *against what was achievable?*" — a fair scoreboard, and a fundamentally different management conversation. The model changed what the organization measures before it changed anything the organization does.

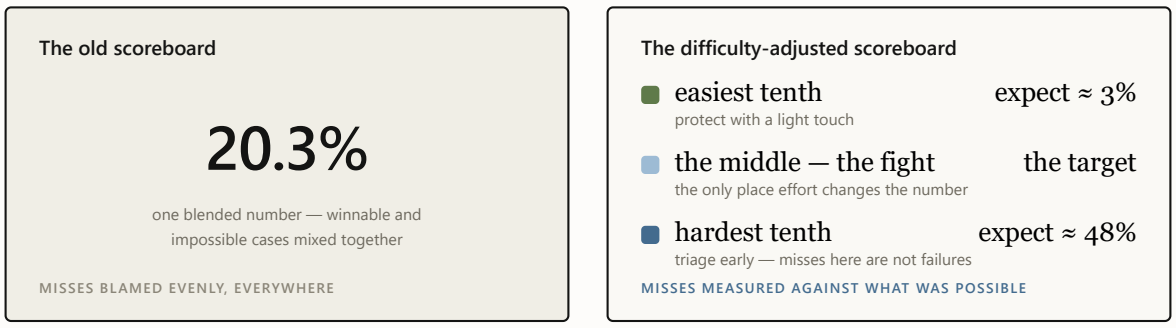


FIGURE 2 Same cases, two scoreboards. The blended number can only go up or down; the difficulty-adjusted one says where to spend effort and whom to stop blaming. This reframe — not any single prediction — is the business product of the forecast layer.

Forecast-first has a quiet second benefit: it forces the data discipline that everything later depends on — strictly as-of features, automated leakage audits, calibration checks — while the stakes are still just a number on a validation report. If the first product were an acting agent instead of a forecast, a data problem would arrive as behavior in production; in a forecast layer it arrives as a number you can catch and fix before anything acts on it.

Restraint, to be clear, is not absence. Inside this pattern, a specialist model fleet reads every case-day under the same as-of and promotion discipline. The

discipline is not a substitute for the machine learning — it is what makes that much machine learning safe to run.

3. Price every claim

Most ML systems fail socially before they fail technically: they say more than they know, someone believes it, and the credibility never recovers. The third rule is to treat claims as things with a price — and to refuse to make any claim whose instrumentation you haven't built.

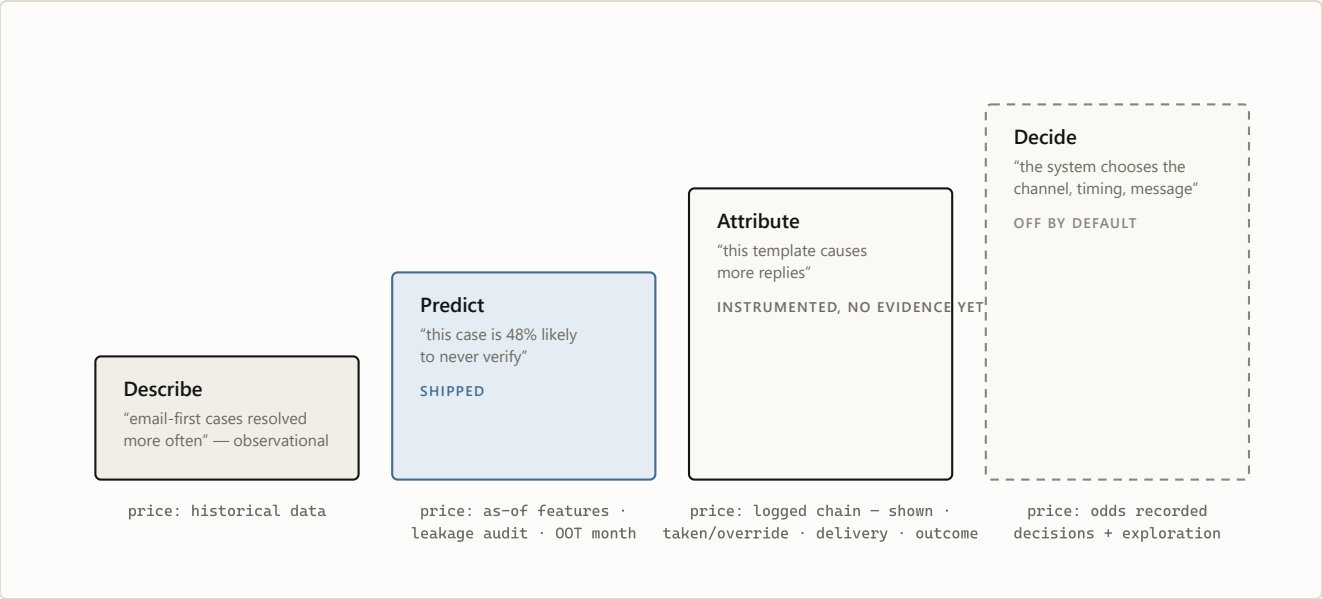


FIGURE 3 The ladder of claims. Each rung is priced in instrumentation, and the price is paid before the claim is made, not after. I hold the line publicly at rung two: correlations like "email-first resolves more often" are reported as observation, never as advice, because selection effects — not wording — may be doing all the work.

Two artifacts make this rule real rather than aspirational. The first is a written **disallowed-claims list** in the architecture record — no causal channel or template claims, no calling the router an autonomous policy, no training a "learned policy" on historical logs and pretending the logs were experiments. The second is a **mechanical promotion gate** for the models themselves.

Three challengers improved the average and weakened the first page operators actually work, so all three were rejected. The written “no” is part of the product.

> *Autonomy is not a capability you install. It is a permission the evidence grants — and the default answer is no.*

4. Let the system earn its next layer

The final rule turns the other three into a trajectory. Authority is granted in stages, each stage gated on evidence the previous stage was designed to produce — and every gate defaults to closed.

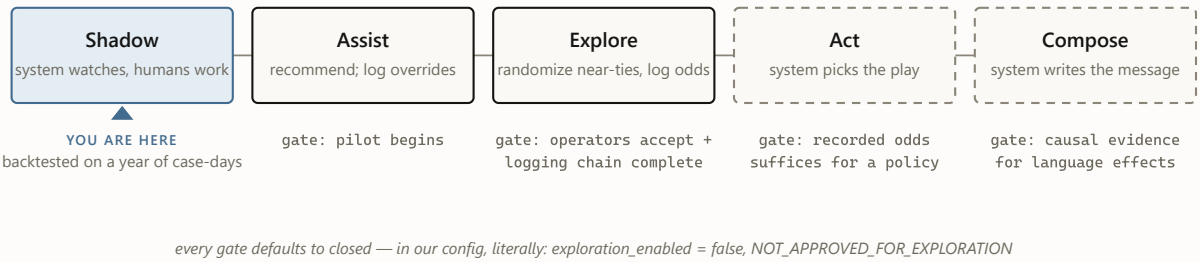


FIGURE 4 The autonomy ladder. Each stage exists to generate the evidence the next stage needs: the pilot logs overrides, overrides calibrate trust, exploration records each option’s odds of being offered, and those recorded odds make a learned policy honest. Skipping a rung doesn’t accelerate the system — it caps how far it can ever be trusted to go.

This is why the system's most capable component is the one that doesn't exist yet. The most capable layers sit highest because they are hardest to grade. The roadmap names the destination; evidence decides the pace.

What restraint buys

The pattern has real costs, and it pays for them. The ledger, honestly kept:

WHAT IT COSTS	WHAT IT BUYS
Slower headline progress — the gate rejects your own improvements, and rejections don't make launch announcements.	Deployability from day one — a system that provably cannot violate policy can enter a regulated workflow while still learning to be useful.
Boring demos — "calibrated forecast plus rules" never wows a room.	Failures that are cheap and legible — a wrong model means a suboptimal ordering, not an incident with a name on it.
Unglamorous work first — logging schemas, leakage audits, and manifests before any intelligence.	A metric the business can act on — difficulty-adjusted targets outlive any particular model version.
Self-correction on the record — every gate rejection and validation miss is written down, forever.	Credibility that compounds — every recorded "no" is what makes the eventual "yes" believable.

FIGURE 5 The trade, side by side. Everything on the left is real and annoying; everything on the right is why the pattern survives contact with a regulated business.

WHEN NOT TO USE THIS PATTERN

The full apparatus is overkill where its costs buy nothing: actions that are cheap and reversible, no compliance surface, outcomes observable within minutes, no human in the loop to protect. Ranking a newsletter's subject lines needs an A/B test, not an evidence-state resolver. The pattern earns its weight precisely where mistakes are expensive, constraints are regulatory rather than aesthetic, and trust — of operators and regulators — is the scarce resource.

The boring version, on purpose

The pattern is one idea applied four ways: rules own permission, models own likelihood, claims stop where instrumentation stops, and every increase in the system's authority must be purchased with evidence. Today that produces forecasts, recommendations, and human review. Tomorrow it can support contact repair, drafted outreach, and controlled auto-send — granted only after logged evidence proves each narrow lane safe, stable, and effective. The path is ambitious *because* the gates are real. Restraint is the launch mechanism, not the destination. ◆

The series. This essay is the design philosophy. Also: the engineering deep dive (architecture, champions, the promotion gate) · the illustrated walkthrough (one case through the machine) · the executive briefing · the Murphy Brown field report.

Internal note written in the style of a public post. All figures are backtested and as-of; correlations are reported as observation, never as causal advice; pilot results are not claimed.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

AFTER THE FIVE PIECES

The operation we are building

Six machine-learning models already read every open case and predict its path; deterministic rules remove unsafe actions; the engine recommends the next move. That claim rests on 2.75 million replayed case-days and zero policy violations — not on a destination slide.

The live workflow records what the system recommended, what a person chose, what was sent, who replied, and how the case ended. That record teaches the machine-learning models which complete plays work, sends an AI researcher after better contact paths, and gives a small language model the evidence to draft the outreach. A narrow lane moves to controlled auto-send only after its logged results prove it safe, stable, and effective.

The first release recommends. The destination learns.

The engine begins by recommending the next move. It ends as an operation that learns how to complete the case.